

M1.1: Multi-Raft Region 概念与分片范围划定

Theory: 水平扩展海量节点的分布式数据切片逻辑。TiKV 不使用哈希分区，而是使用范围分区（Range Partition），将全局 Key 空间切分为大小约为 96MB 的独立 Region，每个 Region 对应一个物理 Raft 共识组。

Details: 这种设计使每个存储节点（Store）在同一时刻运行着数千个独立的 Raft 副本。每个 Raft 组拥有自己独立的 Peer 指针和状态机。通过将全局数据划分为小的 Region，TiKV 可以在多个物理节点之间实现亚秒级的分片重平衡。

Trade-off: 海量 Raft 组会产生显著的背景心跳（Heartbeat）开销，频繁消耗 CPU 算力。TiKV 在空闲 Region 引入“心跳合并与静默（Raft Sleep）”机制，对长期无读写 Region 挂起心跳，降低背景开销。

M1.2: Raft 日志复制与状态机应用

Theory: 保证单 Region 内部多节点强一致性的共识机制。

Details: 当 Leader 接收到写入请求时，它将操作封装为 Raft Log Entry，并将其并行广播给所有 Follower。每个 Peer 在接收到日志后安全追加到本地物理 raftdb，并返回 Ack。一旦 Leader 收到超过半数（Quorum）节点的 Ack，立即标记日志为 Committed 并触发状态机应用，最终将数据提交到 kvdb 中。

Trade-off: 串行的 Raft 日志确认会产生显著的写入延迟。TiKV 实现了“异步提交（Async Commit）”和“单步管道化（Pipelining）”优化，允许日志写入物理磁盘与确认响应在多线程并发并轨处理，将延迟减半。

M1.3: Region 物理分裂 (Split) 与重平衡

Theory: 分布式存储系统防止局部“热点”与大 Region 文件退化的自我演进。

Details: 当单个 Region 的数据尺寸超过阈值（如默认 144MB）或写入键数目过载时，Raftstore 触发 Split 提案。Leader 发起一个特殊的 ConfChange 操作，将原 Region 分裂为两个连续的新 Region，自动更新路由表并通知 Placement Driver (PD)。

Trade-off: 物理分裂会短暂暂停当前 Region 的客户端写入。TiKV 将 Split 细化为无锁元数据转换，并由 PD 动态指挥新 Region 副本的物理迁移，以确保不发生全局流量颠簸。

M1.4: Region 物理合并 (Merge) 时序逻辑

Theory: 回收集群冷数据空洞、精简 Raft 心跳负担的资源整理器。

Details: 当某些 Region 发生大范围物理删除导致数据量萎缩（如小于 20MB）时，PD 会发起 Merge 指令。合并过程极其复杂：两个相邻的 Region 必须先暂停物理写入，利用特定的“Raft Catchup”让两个 Raft 状态机的日志对齐到同一 Log Index，随后在内核原子合并路由。

Trade-off: 合并比分裂更容易引发脑裂风险。TiKV 规定只有相邻的 Region 才能合并，且合并过程必须通过两阶段的共识日志追加硬化，发生网络分裂时立即回滚。

M1.5: Placement Driver (PD) 全局路由调度

Theory: 整个 TiKV 分布式集群的“大脑”与路由指挥部。负责全局时间戳分配（TSO）、元数据存储与副本平衡调度。

Details: PD 采用 etcd 内核构建强一致性元数据管理。TiKV 节点通过周期性的 Heartbeat 向 PD 上报自身的磁盘占用、CPU 负载以及 Region 分布情况。PD 根据这些指标生成调度指令，指挥 Region 在不同 Store 之间迁移，确保整个物理集群的水位平衡。

Trade-off: PD 是集群的单点性能瓶颈（SPOF）。为了扩展性，客户端在冷启动时从 PD 拉取路由表并缓存到本地，后续读写直接与对应的 TiKV 节点交互，仅在缓存路由失效时重新查询 PD。

M2.1: Percolator 分布式两阶段提交 (2PC)

Theory: 基于单行事务悲观锁，实现无全局事务协调器的去中心化行级强一致性事务。

Details: 客户端首先向 PD 申请 'start_ts' 时间戳。Prewrite 阶段：选择一个 Key 作为 Primary Key，向 RocksDB 的 Lock CF 写入锁元数据，其余 Secondary Keys 则写入指向 Primary Key 锁的引用。Commit 阶段：先向 PD 获取 'commit_ts'，清除 Primary Key 的 Lock 并向 Write CF 写入提交版本；Primary 提交成功后，Secondary Keys 异步并行解锁。

Trade-off: 在 Prewrite 冲突时（如发现已有比自己 start_ts 更晚的锁），事务必须回滚。这使得在高并发写冲突下，Percolator 事务的回滚率极高，需要引入悲观锁模式以降低冲突冲突。

M2.2: Lock CF 锁管理与并发冲突控制

Theory: 承载 Percolator 分布式锁元数据存储的物理隔离列族。

Details: RocksDB 内部使用多个 Column Families (CF) 实现逻辑分区。Lock CF 记录了当前未提交事务所持有的锁。锁内记录了锁的类型（Put/Delete）、'start_ts'，以及最重要的“Primary Key 指针”信息。这使得其他事务在读取到此锁时，能迅速顺藤摸瓜找到 Primary Key 以探测主事务是否已提交。

Trade-off: 频繁的锁读写会导致 Lock CF 产生大量的 Tombstones（删除标记）。为此 TiKV 对 Lock CF 的 Compaction 进行针对性调优，强制高频压实以防止读性能滑坡。

M2.3: MVCC (多版本并发控制) 读写路径

Theory: 实现非阻塞读取的存储层机制。读操作不加锁，直接根据时间戳读取历史版本。

Details: 在读取数据时，客户端携带 'read_ts'。TiKV 首先检查 Lock CF 在 'read_ts' 之前是否有未提交的锁。若无，则去 Write CF 检索在 'read_ts' 之前最新的提交版本 'commit_ts'。找到后，通过对应的 'start_ts' 重定向去 Default CF 提取真实的物理数据值。

Trade-off: MVCC 保证了读写不冲突，但会引发空间膨胀。TiKV 在物理 Compaction 时，由底层的 GC 线程根据全局的 Safe Point（安全清理时间戳）强制擦除过期的旧版本数据。

M2.4: 乐观事务与悲观事务锁差异

Theory: 应对不同并发场景的冲突防范折衷。

Details: 乐观事务在 'await' 期间不在 TiKV 侧加锁，全部缓存在客户端内存中，直到最后 Commit 时才进行 2PC 写入；悲观事务则在运行中执行 SQL（如 'SELECT FOR UPDATE'）时，立即在 TiKV 侧写入悲观锁，直接卡死其他竞争者的 Prewrite。

Trade-off: 乐观事务适合写冲突低的场景，开销极低；悲观事务在写冲突高的场景下虽然会产生显著的加锁网络延迟，但能规避 Percolator 后期大范围回滚的灾难性开销。

M2.5: 分布式死锁检测算法

Theory: 解决悲观锁模式下，多个分布式节点之间由于循环等待锁而卡死的自我治愈。

Details: 每个 TiKV 节点内部拥有一个本地死锁检测器。当发生锁等待时，构建本地等待关系图 (Wait-For Graph)。为了解决跨节点的环路，TiKV 将等待关系上报给 Leader 节点或通过分布式死锁检测环路探测；如果发现 'Txn A -> Txn B -> Txn C -> Txn A' 的闭环，立即向被套死事务发送 Abort 错误。

Trade-off: 高并发下死锁检测会引入一定的锁延迟和检测流量损耗。TiKV 默认配置死锁超时自动回退机制作为双重保险。

M3.1: RocksDB 双存储引擎集成

Theory: 将底层物理业务数据与共识日志进行物理隔离存储以优化 IO。

Details: 单个 TiKV 节点内跑着两个 RocksDB 实例。‘raftdb’ (Raft 引擎) 专门用于顺序写入和消费 Raft consensus log; ‘kvdb’ (KV 引擎) 存储真实的业务数据及 MVCC/Lock 状态。

Trade-off: 两套引擎意味着两套 WAL 和各自的 MemTable, 增加了约一倍的内存基础开销。为榨干性能, TiKV 对 raftdb 使用极其激进的内存和刷盘配置, 甚至在最新版本中可以用自研的 ‘Raft Engine’ 代替 rocksdb raftdb。

M3.4: Block Cache 与 Bloom Filter 检索加速

Theory: 规避 LSM-Tree 多层检索导致的物理 I/O 读取开销的硬加速。

Details: RocksDB 寻找某个 Key 时, 需要自顶向下遍历各层 SSTable。为了防止多次物理读盘, 每个 SST 文件的尾部附加了布隆过滤器 (Bloom Filter), 能以 $O(1)$ 的开销判定 Key “绝对不存在”; Block Cache 负责缓存已解密和读取的 SST 数据块, 加速热点访问。

Trade-off: 布隆过滤器和块缓存都需要占用大量的物理 RAM。Cilium-style 内存管理要求必须在启动时精心分配 Block Cache 占比, 避免物理 OOM。

M3.5: RocksDB 写入放大调优折中

Theory: 读吞吐、写吞吐和物理空间回收效率之间的经典理论限制。

Details: 较少的 Compaction 能提高写入吞吐, 但会导致大量垃圾文件堆积 (空间放大) 和多层扫描 (读放大); 频繁的 Compaction 则反之。

Trade-off: TiKV 对数据类型进行分 CF 配置。对于 Default CF, 采用 Leveled Compaction 保证读性能; 对于 Lock CF, 采用更激进的压实参数, 力求零 Tombstone 积压。

M4.3: Chunk 内存紧凑布局与传输

Theory: 消除列式计算与网卡传输之间序列化损耗的高能内存块结构。

Details: 协处理器在内存中使用连续的、对齐的列式数组 (Col-Buffer) 组织 Chunk。一系列的数据在物理内存中完全连续。当数据过滤完毕需要通过 gRPC 发回时, 几乎不需要任何格式转换, 可以直接通过套接字进行 zero-copy 物理数据块倾倒。

Trade-off: 这种紧凑布局要求写入逻辑高度规整。在处理稀疏数据或包含大量 Null 值的字段时, 需要引入额外的 Bitmap 标志位, 增加了部分物理索引复杂度。

M3.2: WAL 刷盘与 MemTable 无锁写入

Theory: RocksDB 内部确保数据持久化与高并发内存写入的经典流水线。

Details: 当数据写入 RocksDB 时, 首先顺序追加到磁盘的 Write-Ahead Log (WAL) 以防断电丢失, 随后写入内存的 ‘MemTable’。MemTable 底层是高度优化的无锁跳表 (SkipList), 允许多个线程并发读写。

Trade-off: 强行同步刷盘 (fsync) WAL 会极大降低写入性能。TiKV 在 raftdb 上通常开启 Group Commit, 而在 kvdb 上默认配置为异步刷盘, 依靠 Raft 副本的多数派内存拷贝来保证整体容灾安全性。

M4.1: Coprocessor 物理算子下推原理

Theory: 减少海量分布式数据在网络传输中产生的严重带宽瓶颈的近源计算。

Details: 如果客户端执行 ‘SELECT COUNT(*) FROM users WHERE age > 18’, 传统架构必须拉取所有行数据到主客户端过滤。TiKV 的 ‘Coprocessor’ (协处理器) 允许将过滤 (Filter)、聚合 (Aggregation) 的算子打包为 DAG 下推至各存储节点本地, 就地读取 RocksDB 并执行计算, 最后仅返回结果计数值。

Trade-off: 算子下推会大幅消耗存储节点的 CPU 资源。如果大量慢 SQL 执行下推, 会导致主 Raft 心跳被抢占而引发心跳超时, 必须合理配置 Coprocessor 的多线程调度优先级。

M4.4: CPU SIMD 指令集编译加速

Theory: 榨干存储节点物理硬件算力的高级编译器优化。

Details: TiKV 采用 Rust 开发。在编译协处理器组件时, 通过配置目标 CPU 指令集 (如启用 AVX-512、Neon), 使 Rust 编译器能够自动将向量化 Chunk 计算循环翻译为 SIMD 物理并行指令, 实现一个 CPU 周期并发比较 8 个 i64 数据。

Trade-off: SIMD 强绑定特定的宿主物理 CPU 架构。如果编译出的二进制分发到老旧 CPU 上运行, 会直接触发 ‘SIGILL’ (非法指令) 异常崩溃。必须针对云原生环境做自适应指令探测。

M3.3: SSTable 分层与 Compaction 整理机制

Theory: 解决 LSM-Tree 存储引擎中, 数据删除与更新导致的物理碎屑及读放大。

Details: 随着 MemTable 满, 数据被 Flush 到 Level 0 SSTable 磁盘文件。RocksDB 自动触发后台 Compaction: 将 Level N 的 SST 文件与 Level $N + 1$ 的文件进行多路归并排序合并, 擦除被覆写或打上了 Delete 标记 (Tombstone) 的旧键值, 下沉到更深层级。

Trade-off: Compaction 会消耗海量的物理磁盘 I/O 带宽 (写放大)。在高吞吐写入下, Compaction 与前台写请求争抢网卡/磁盘, 会导致严重的读写延迟抖动, 需要进行严格的写速率限流。

M4.2: Vectorized 向量化算子执行

Theory: 规避传统行级迭代器 (Volcano Model) 高频虚函数调用和 CPU 分支预测失败的现代引擎设计。

Details: 协处理器内部全面采用向量化执行引擎。每个算子不再单行返回, 而是每次批量处理并返回一个包含数千行数据的 ‘Chunk’。例如, 执行 ‘age > 18’ 过滤时, 在一个紧凑的循环中对一个 Chunk 的整个 age 数组执行比较。

Trade-off: 向量化能将 CPU L1 缓存利用率拉满并触发 CPU 自动向量化编译 (SIMD)。缺点是增加了算子之间传递临时 Chunk 的内存分配开销, 需要引入内存复用池。

M4.5: gRPC 多路复用与心跳检测

Theory: 百万级网络 IO 高并发下的连接复用与生命周期探测。

Details: TiKV 节点之间使用 gRPC 构建通信骨架。通过 HTTP/2 协议实现单条 TCP 连接上的多路复用 (Multiplexing), 避免频繁建立连接的系统开销。同时, gRPC 内置 Keepalive 心跳探测机制, 用于在物理光纤或网络接口异常时快速断开并报错。

Trade-off: HTTP/2 的多路复用单连接高频吞吐下可能会遇到 TCP 层的队头阻塞 (Head-of-Line Blocking) 漏洞。高频通信的节点间会配置多条独立的物理连接以均摊开销。

M5.1: Raft 消息流异步 Pipeline 处理

Theory: 彻底榨干网络 IO 与磁盘写入并轨性能的多线程并发流水线。

Details: 在 Raftstore 内部, Raft 消息处理被解耦为三个核心阶段的 Pipeline: 网络接收接收 物理磁盘写入 (日志落盘) 内存应用 (状态机执行)。每个阶段由独立的线程池 (Raft worker、Apply worker) 处理, 中间通过无锁 Ring Buffer 通道连接。

Trade-off: Pipeline 极大提升了吞吐, 但在高并发下会引发排队延迟。当磁盘写入慢时, Apply worker 会因为拿不到提交日志而处于饥饿状态, 需要在入口处配置限流。

M5.2: 节点间零拷贝 Snapshot 传输

Theory: 应对新节点加入或故障恢复时, 数 GB 级超大 Region 数据迁移的零 CPU 损耗传输。

Details: 当一个 Region 副本缺失需要同步 Snapshot 时, Leader 会在 kvdb 侧直接生成对应 Key 范围的 RocksDB SST 文件。在传输时, 通过 'sendfile' 系统调用将该文件直接从磁盘缓存发送到网络套接字, 完全绕过用户态内存, 降温 CPU 损耗。

Trade-off: 生成 Snapshot SST 文件需要一次全量内存/磁盘扫描, 这会引发 RocksDB 局部读写性能波动。TiKV 允许通过限速参数严格限制每秒传输的 Snapshot 带宽。

M5.3: Backpressure 背压限流自愈

Theory: 存储底座在物理资源耗尽时主动放慢上层写入速率的自卫机制。

Details: 当 RocksDB 的 Compaction 严重滞后, 导致 Level 0 SSTable 文件积压突破阈值 (如 30 个) 时, 前台写入会有被卡死的危险。TiKV 的背压机制会立刻检测到该指标, 通过主动延缓向 Leader 回应 Ack 或减慢 gRPC 读速率, 将压力向上传导至客户端, 强制业务端限流, 防止物理 OOM 崩溃。

Trade-off: 限流会导致业务端出现短暂的写入超时或延迟突增, 但这是保证数据库不产生死锁或 OOM 强退的最优防御措施。

M5.4: TiKV 网络分区故障自愈

Theory: 分布式系统面对网络恶劣断开 (脑裂) 时的共识防御。

Details: 当集群发生网络分区, 分裂为两个孤立子集时, 包含多数派 (Quorum) 节点的子集能够正常完成 Raft 共识并继续服务; 而少数派子集因为无法获得多数 Ack, 所有写入自动卡死。当网络恢复后, 少数派通过同步 Leader 的最新日志进行自我修正与对齐。

Trade-off: 脑裂可能导致少数派在分区期间提供过期的读数据 (Stale Read)。必须开启 “租约读 (Lease Read)” 或 “线性一致性读 (Read Index)” 来保证每次读取必须去 Leader 确认租约。

M6.1: Failpoint 注入与混沌工程测试

Theory: 验证强一致性系统在极端物理故障下不丢失数据、不发生状态错乱的防御性质量校验。

Details: TiKV 内部嵌入了 'fail-rs' (Failpoint) 框架。在代码的关键并发路径 (如磁盘写入前、网络发送时、锁释放期间) 埋入控制点。在自动化测试中, 通过环境变量或 RPC 接口动态激活这些埋点, 模拟 “突然磁盘满”、“内核挂起”、“网络丢包”, 检验状态机能否自愈。

Trade-off: 埋点在编译时仅在 Debug/Test 模式下生效。生产环境编译 (Release) 会通过条件编译将 Failpoint 代码完全剔除, 确保零运行时性能损耗。

M6.2: RocksDB 运行期监控指标遥测

Theory: 数据库稳定运行的物理生命指标诊断。

Details: 遥测探针重点监控 RocksDB 的关键指标: SST 文件数、Write Stall 发生频率 (写入挂起)、MemTable 内存占用、Block Cache 命中率以及 Compaction 吞吐。这些指标通过 Prometheus 接口实时暴露给 Grafana 面板。

Trade-off: 频繁读取 RocksDB 的 internal 状态 (如 'GetProperty') 会有一定的原子锁竞争开销。TiKV 采用定时器异步轮询获取指标, 避免影响前台写入。

M6.3: 分布式 Tracing 链路诊断

Theory: 跟踪单条 SQL 从客户端发起、经过 PD、通过 gRPC、进入 Raftstore 直到写入 RocksDB 的微观执行生命周期。

Details: TiKV 集成了 'minitrace' 追踪框架。当执行一条慢事务时, 生成的 Trace ID 跨 RPC 边界传递。每个处理线程记录自身的 Span 区间 (如 'raftstore_commit_time'、'rocksdb_write_time'), 最终聚合为完整的火焰图展示延迟瓶颈。

Trade-off: 全量 Tracing 会产生庞大的追踪日志并消耗网卡带宽。生产环境只采用概率抽样 (如 0.1% 抽样) 或仅在检测到执行时间超出阈值 (慢查询) 时才落盘 Trace。

M6.4: Card 28 Fallback Title

Placeholder text for compiler robustness.

Zone T1: TiKV Crate APIs

```
tikv::Storage::async_write(), raftstore::store::RaftCmd(),
coprocessor::Endpoint::handle_request(), txn::Prewrite(),
txn::Commit(), rocksdb::DB::write_cf(), rocksdb::DB::get_cf()
```

Zone T2: System Data Faults

```
percolator_lock_conflict, write_stall_backpressure,
raft_log_lag_desync, network_partition_split_brain,
rocksdb_compaction_io_saturation, distributed_deadlock_detected,
distributed_deadlock_detected
```